

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

Лабораторная работа №3
по дисциплине
"Web-разработка: Frontend"

Семестр IV

Выполнил:
студент
Бровкин Аким Алексеевич
гр. J3210
ИСУ 465282

Санкт-Петербург
2026

1 Цель работы

Целью данной работы является изучение компонентного подхода и миграция образовательной платформы на современный JavaScript-фреймворк Vue 3 с настройкой клиентской маршрутизации и асинхронного взаимодействия с REST API.

В ходе работы необходимо было разделить монолитный код на переиспользуемые модули, настроить роутер для создания Single Page Application и заменить нативные сетевые запросы на использование библиотеки Axios.

2 Задачи работы

1. Инициализация проекта с использованием сборщика Vite.
2. Настройка Vue Router для реализации клиентской маршрутизации и защиты приватных страниц.
3. Выделение повторяющейся логики (авторизация, управление темами оформления) в независимые модули — Composables.
4. Разработка UI-компонентов для устранения дублирования разметки.
5. Перевод взаимодействия с локальным моковым API на использование HTTP-клиента axios.
6. Решение проблем с кэшированием статики и интеграция SVG-спрайтов с поддержкой динамического изменения иконок.

3 Описание реализации

3.1 Инициализация и структура проекта

Для сборки проекта я использовал бандлер Vite. Логика приложения была строго структурирована: страницы вынесены в директорию `views`, переиспользуемые элементы интерфейса — в `components`, а общие функции — в `composables`.

3.2 SPA-маршрутизация

Для обеспечения навигации без перезагрузки страниц я внедрил `vue-router`. Был создан конфигурационный файл маршрутов, в котором реализован глобальный хук `beforeEach`. Данный хук проверяет наличие сессии пользователя в `localStorage` и предотвращает несанкционированный доступ к страницам личного кабинета и поиска курсов, перенаправляя неавторизованных пользователей на страницу входа.

3.3 Компоненты и Composables

Я применил подход Composition API, выделив бизнес-логику в независимые файлы.

- **useAuth.js:** инкапсулирует методы `login`, `register` и `logout`, а также хранит глобальное реактивное состояние текущего пользователя.
- **useTheme.js:** отвечает за переключение атрибута `data-theme` в корневом элементе и сохранение выбора пользователя.

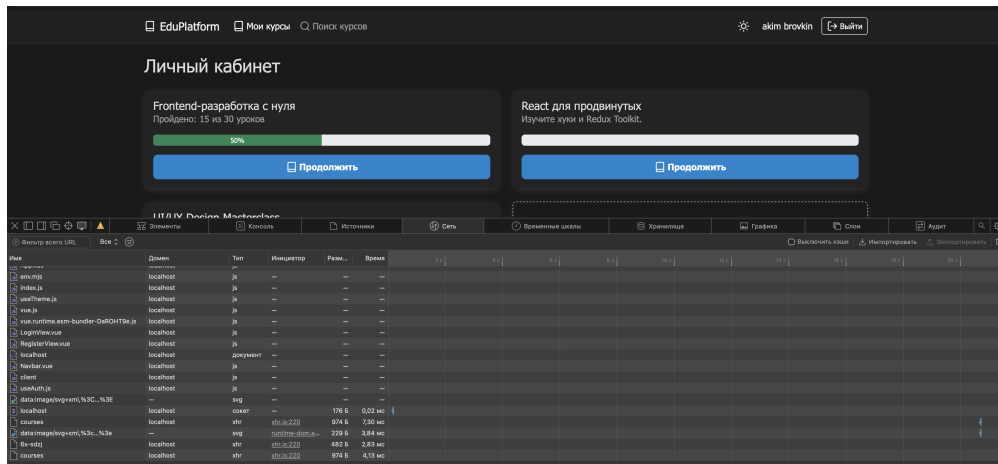


Рис. 1: Сетевые запросы при SPA-навигации: HTML-документ не перезагружается

Интерфейс был декомпозирован. Например, карточка курса (`CourseCard.vue`) теперь принимает данные о курсе через `props` и генерирует события при попытке пользователя записаться на курс, делегируя логику API родительскому компоненту.

3.4 Взаимодействие с API через Axios

Все запросы к локальному `json-server` были переписаны с использованием `axios`. Загрузка данных (например, списка всех курсов и профиля пользователя) происходит асинхронно в хуке жизненного цикла `onMounted`. Работа с `axios` позволила сократить объем кода за счет автоматического парсинга JSON-ответов.

3.5 Динамические SVG-спрайты

Особое внимание было уделено работе с иконками. Для исключения проблем с асинхронной загрузкой графики и ограничениями CORS в некоторых браузерах, код `sprite.svg` был интегрирован непосредственно в стартовый `index.html`. В компонентах Vue реализована динамическая привязка атрибутов для иконок.



Рис. 2: Отображение темной темы с динамически измененной SVG-иконкой

4 Примеры реализации

Ниже приведен фрагмент кода компонента навигации, демонстрирующий использование Composition API и динамическое связывание атрибутов (переключение иконки темы в зависимости от состояния):

Листинг 1: Использование Composables и динамических атрибутов в `Navbar.vue`

```
<template>
  <nav class="navbar custom-navbar">
    <button @click="toggleTheme" class="theme-toggle-btn text-light">
      <svg class="icon">
        <use :href="theme === 'dark' ? '#icon-sun' : '#icon-theme'">
      </use>
    </button>
  </nav>
```

```

        </svg>
      </button>
      <span class="text-light">{{ currentUser?.fullName }}</span>
      <a href="#" @click.prevent="logout" class="btn btn-outline-light"
        >

    </a>
  </nav>
</template>

<script setup>
import { useAuth } from '../composables/useAuth'
import { useTheme } from '../composables/useTheme'

const { currentUser, logout } = useAuth()
const { theme, toggleTheme } = useTheme()
</script>

```

5 Заключение

В результате выполнения лабораторной работы проект был успешно мигрирован на Vue 3. Использование компонентного подхода, клиентского маршрутизатора и выделения бизнес-логики в `composables` позволило значительно повысить читаемость, масштабируемость и модульность кодовой базы. Приложение приобрело свойства Single Page Application, что улучшило общий пользовательский опыт за счет мгновенной навигации и устранения лишних сетевых загрузок.